VanTrade

Final Architecture Report

Course: Software Architecture

Date: April 13, 2026

Repository: vantrade (pnpm monorepo)

By

Thanwat Tantijaroesin 6610545294

1. Project Overview

VanTrade is a multi-tenant algorithmic trading strategy marketplace. Strategy Providers publish parameterized trading Blueprints (RSI mean-reversion or ICT/Smart Money Concepts strategies); Testers subscribe and execute them automatically against their personal Alpaca Paper Trading accounts, choosing which asset to run each strategy on; Admins verify Blueprints before they appear on the marketplace.

Scope is intentionally bounded to paper trading — no live money, no HFT, no ML model training.

1.1 System at a Glance

The monorepo contains three packages:

- apps/api — NestJS REST API (port 4000): owns all business logic, Prisma, Alpaca SDK
- apps/web — Next.js 14 frontend (port 3000): UI only, no business logic
- packages/types — Shared Zod schemas and TypeScript interfaces (@vantrade/types)

Build orchestration: pnpm workspaces + Turborepo

Database: PostgreSQL via Prisma 5

Broker: Alpaca Paper Trading API

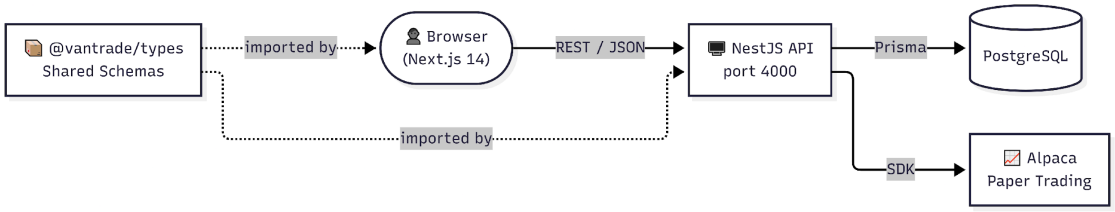

Figure 1 — System Overview

2. System Requirements

2.1 Functional Requirements

#	Requirement	Implementation
FR-1	Three distinct user roles with own responsibilities, permissions, and access control	TESTER, PROVIDER, ADMIN enforced via RolesGuard + @Roles() on every protected endpoint
FR-2	Each role performs at least one CRUD operation	TESTER: subscribe/unsubscribe/toggle; PROVIDER: blueprint create/read/update/delete; ADMIN: verify blueprint, list all users, assign user roles
FR-3	Secure login/logout, user authentication	JWT via Passport.js; bcryptjs password hashing; JwtAuthGuard on all protected routes
FR-4	Role-based authorization and access control	Server-side RolesGuard — client cannot escalate privileges regardless of what it sends
FR-5	From-scratch development	Full custom implementation; no boilerplate app templates used

2.2 Non-Functional Requirements

Attribute	Requirement	Mechanism
Security	Broker credentials must never be stored in plaintext	AES-256-GCM encryption at rest; decryption in-memory only at order time
Security	API surface must be rate-limited	@nestjs/throttler — 10 req/s burst, 200 req/min sustained
Reliability	One subscription failure must not stop others	Promise.allSettled in HeartbeatService
Auditability	Trade history must be immutable	Append-only TradeLog — no update or delete paths exist
Maintainability	Broker swap must require zero domain changes	Ports & Adapters — new adapter + one DI rebinding
Testability	Trading calculations must be verifiable in isolation	Pure functions in trading.engine.ts — no mocks needed
Type Safety	No type drift between API and frontend	Shared Zod schemas in @vantrade/types

3. Architecture Characteristics

VanTrade's architecture was designed around several key quality attributes driven by the domain requirements of a financial trading platform.

3.1 Security

Broker API credentials are encrypted with AES-256-GCM before database storage. The encryption key is scrypt-derived from an environment variable. Decrypted values are computed in-memory at order time only and passed as function call arguments — never stored on adapter instances. The API surface is protected with rate limiting (10 req/s burst, 200 req/min sustained) and every endpoint enforces JWT authentication and role-based access control server-side.

3.2 Reliability

The heartbeat execution loop processes all active subscriptions using Promise.allSettled. This means a single subscription failure — such as invalid credentials, a revoked API key, or a network timeout — cannot halt execution for other users. Each subscription runs in its own isolated try/catch block.

3.3 Auditability

TradeLog rows are append-only. No update or delete methods exist in the repository. Every execution outcome — filled order, hold, or error — is persisted as a new row. This immutable ledger enables users to independently verify their trade history and P&L, and powers the heartbeat's trade-side alternation logic without requiring mutable state.

3.4 Maintainability

The Ports & Adapters pattern isolates broker-specific code in a single file (alpaca.adapter.ts). Swapping brokers requires writing one new adapter and changing one DI binding — zero changes to domain logic. The Repository pattern similarly isolates Prisma access; the ORM can be replaced without touching services. Ten feature modules enforce vertical separation.

3.5 Testability

All trading calculations are implemented as pure functions in trading.engine.ts. They take primitive inputs and return deterministic outputs with no side effects, making them testable without mocking any infrastructure. Service-level tests mock repositories; controller-level tests use supertest against the full NestJS app.

3.6 Type Safety

Shared Zod schemas in packages/types serve as the single source of truth for data contracts. The API validates incoming request bodies and query params with ZodValidationPipe; the web app validates API responses at runtime. TypeScript's z.infer derives static types from schemas, giving end-to-end type safety from the database model to React components.

4. Architecture Design

4.1 Hexagonal Architecture (Ports & Adapters)

The trading subsystem enforces strict three-layer separation across all broker-related logic.

Layer	Location	Responsibility
Domain	apps/api/src/trading/trading.engine.ts	Pure functions only — RSI, SMA, signal generation, P&L
Port	packages/types/src/interfaces/IBrokerAdapter.ts	Interface contract that HeartbeatService depends on
Adapter	apps/api/src/trading/broker/alpaca.adapter.ts	Concrete Alpaca SDK implementation — the only file allowed to import the SDK

Domain layer functions — RSI Strategy:

Function	Purpose
calculateRSI(prices, period)	Wilder's smoothing RSI
calculateSMA(prices, period)	Simple Moving Average
generateSignal(rsi, buyThreshold, sellThreshold)	Returns TradeSignal.BUY / SELL / HOLD
calculatePnL(entry, exit, qty, side)	Realised profit/loss for a closed position
calculateUnrealisedPnL(entry, current, qty)	Mark-to-market P&L for an open position

Domain layer functions — ICT / Smart Money Concepts Strategy:

Function	Purpose
aggregateBars(bars, n)	Resamples M5 bars into M15, H1, etc.
detectSwingPoints(bars, lookback)	Identifies local swing highs and lows
detectMarketStructure(bars, swings)	Derives BULLISH / BEARISH / NEUTRAL bias via BOS/CHoCH
detectOrderBlock(bars, type)	Finds the most recent valid OB entry zone
detectFairValueGap(bars, type)	Finds the most recent unmitigated FVG
classifyPriceZone(price, high, low)	Returns PREMIUM / DISCOUNT / AT_EQUILIBRIUM
hasLiquiditySweep(bars, level, type)	Detects stop-hunt wick past a swing level
checkLimitOrderFill(bar, position)	Simulates TP/SL fill in backtests
generateIctSignal(ctx)	Full A+ setup checklist — all five ICT conditions must pass

The DI binding is declared in apps/api/src/trading/trading.module.ts as: { provide: 'IBrokerAdapter', useClass: AlpacaAdapter }. To swap brokers, write a new adapter class and rebind the token — zero changes to domain logic or the heartbeat.

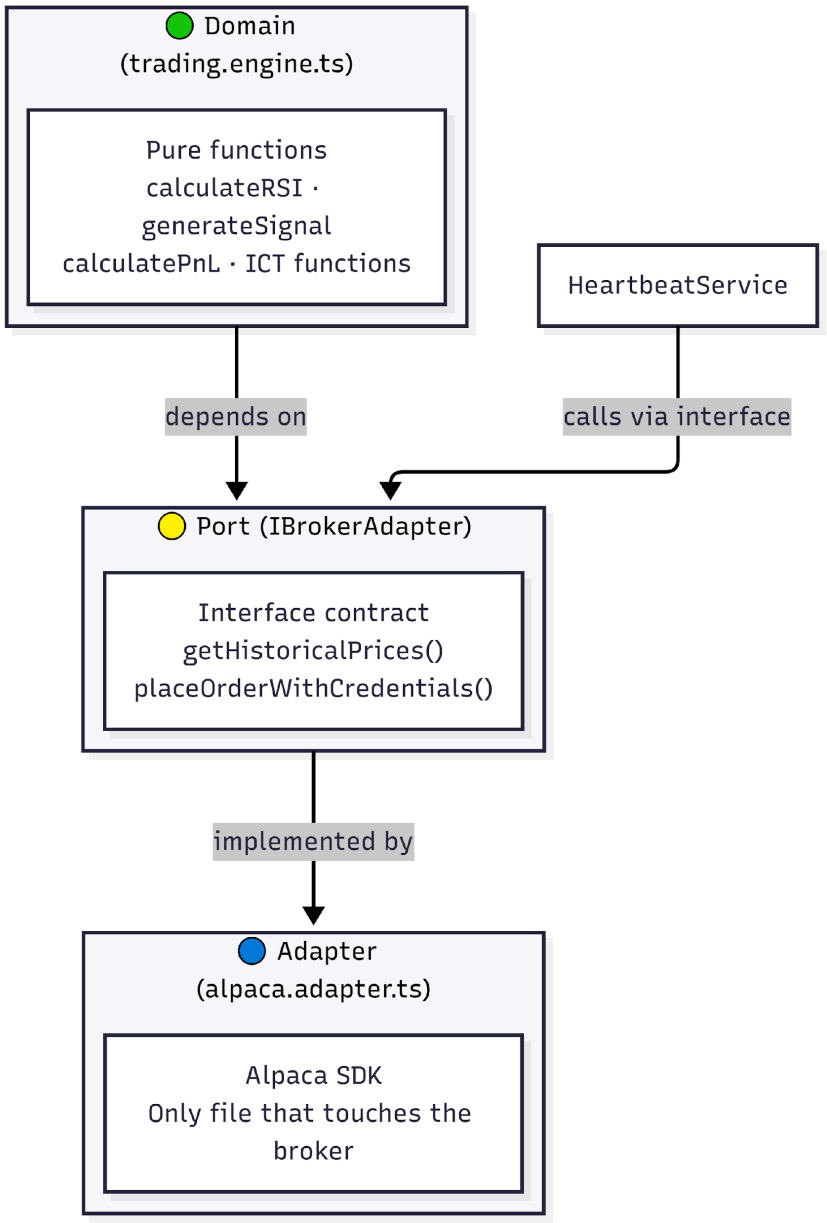

Figure 2 — Hexagonal Architecture (Ports & Adapters)

4.2 Repository Pattern

Every feature module has exactly one *.repository.ts that is the sole file permitted to import PrismaService. No controller or service touches the database directly. PrismaModule is declared @Global() so PrismaService is available to all repositories without explicit imports.

Repository	Key Methods
AuthRepository	findByEmail, findById, create
BlueprintsRepository	CRUD, setVerified, findAllVerified, findByAuthor
SubscriptionsRepository	findByUser, findAllActive, setActive, findExisting
ApiKeysRepository	upsert, findByUser, delete
TradeLogsRepository	create, findBySubscription, getStats, findLatestTradeSideBySubscription

4.3 NestJS Feature Modules (Thin Controller Pattern)

Ten feature modules each follow the same vertical slice structure:

- controller.ts — HTTP boundary: validate input, call service, return result
- service.ts — Business logic and orchestration
- repository.ts — Database access (Prisma only)
- module.ts — DI wiring

The Thin Controller Rule: controllers do exactly three things — validate with ZodValidationPipe, call one service method, return the result. No if/else, no Prisma, no SDK calls inside controllers.

Module	Role
AuthModule	JWT login/register, Passport strategy
BlueprintsModule	Marketplace CRUD, admin verification gate, backtest
SubscriptionsModule	Subscribe/unsubscribe, toggle active, stats
TradingModule	Exports IBrokerAdapter DI token
HeartbeatModule	Background cron execution engine
EncryptionModule	AES-256-GCM key encryption/decryption
ApiKeysModule	Broker credential management
MarketDataModule	Real-time price/bars endpoint for UI
TradeLogsModule	Exports TradeLogsRepository
PrismaModule	Global database connection

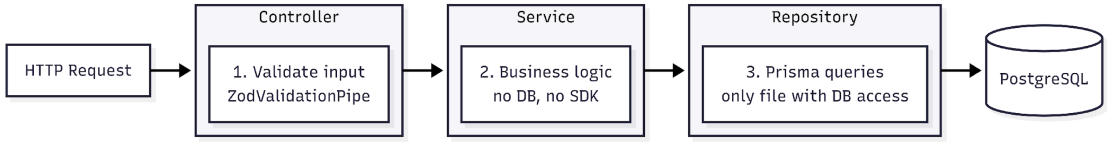

Figure 3 — Request Pipeline (Controller → Service → Repository)

4.4 Heartbeat Execution Loop

File: apps/api/src/heartbeat/heartbeat.service.ts | Schedule: @Cron('*/60 * * * * *') — every 60 seconds

The heartbeat is the core runtime engine. On each tick it processes all active subscriptions in parallel with Promise.allSettled, so a single failure does not block others.

Per-subscription processing pipeline:

- Validate blueprint parameters (Zod)
- Check market hours — Stocks: 9:30 AM–4:00 PM ET weekdays; Crypto: 24/7
- Check timeframe boundary alignment (5Min, 15Min, 1Hour, 1Day)
- Fetch (rsiPeriod + 1) OHLCV bars via IBrokerAdapter
- calculateRSI() — pure domain function
- generateSignal() — pure domain function
- Smart side resolution: read last executed side from TradeLog, enforce BUY→SELL→BUY alternation
- Decrypt user API key via EncryptionService (in-memory only)
- placeOrderWithCredentials() via IBrokerAdapter
- Persist TradeLog (append-only, including HOLDs)

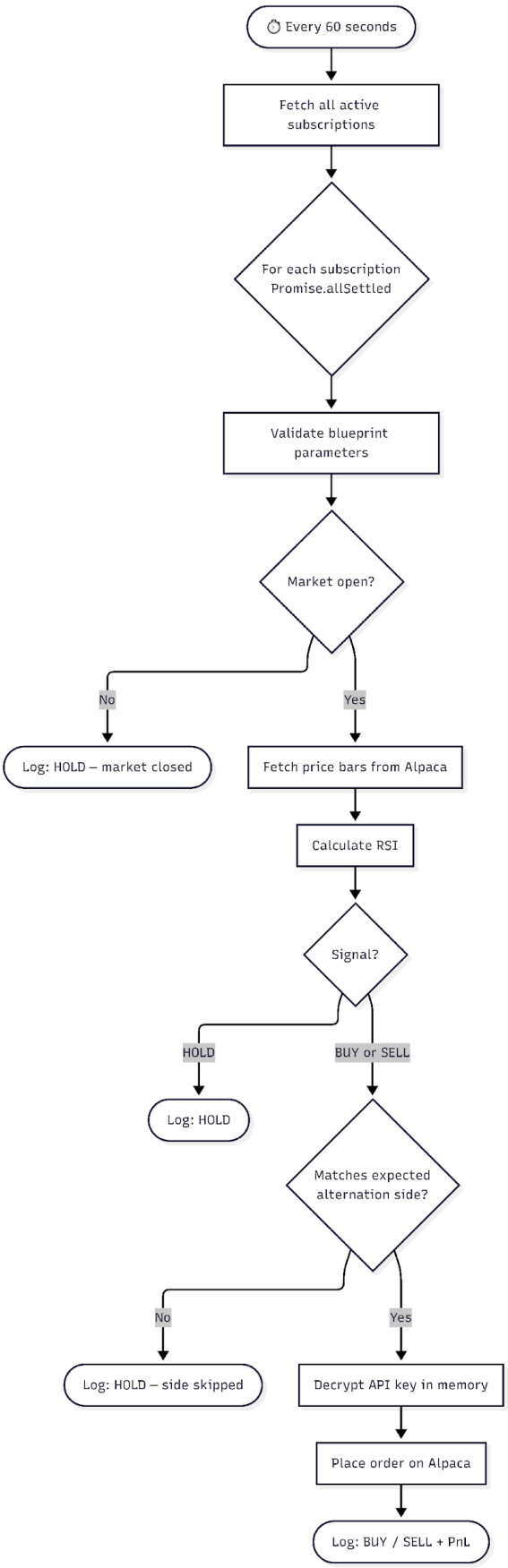
Every 60 seconds
Fetch all active subscriptions
For each subscription Promise.allSettled
Validate blueprint parameters
Market open?
No
Yes
Log: HOLD — market closed
Fetch price bars from Alpaca
Calculate RSI
Signal?
HOLD
BUY or SELL
Log: HOLD
Matches expected alternation side?
No
Yes
Log: HOLD — side skipped
Decrypt API key in memory
Place order on Alpaca
Log: BUY / SELL + PnL

Figure 4 — Heartbeat Execution Flow

4.5 Shared Type Package (@vantrade/types)

All Zod schemas and TypeScript interfaces live in packages/types/src/. Both apps/api and apps/web import from this package. Zod schemas serve double duty:

- API layer: ZodValidationPipe validates incoming request bodies and query params
- Web layer: base.ts API client parses and validates API responses at runtime

A single schema change propagates to both apps at compile time. TypeScript's z.infer derives static types from schemas, giving end-to-end type safety from database model to React component.

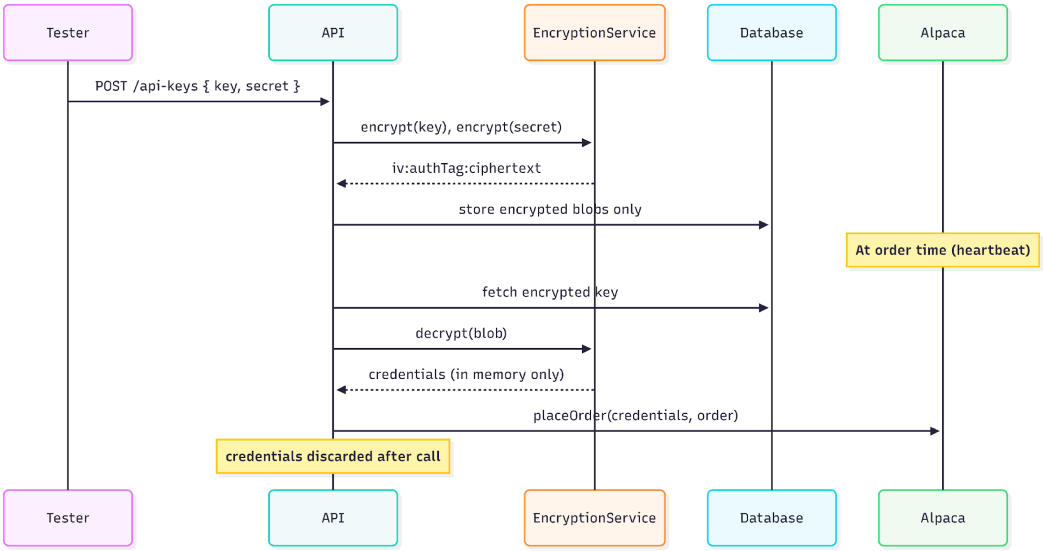

Figure 7 — Credential Encryption & In-Memory Decryption Flow

4.6 CodeCharta Static Analysis as a Fitness Function

CodeCharta v1.142.0 was used as a continuous architectural fitness function. Two metric sources are merged into a single map: rawtextparser produces rloc (real lines of code) showing file size/complexity, and gitlogparser produces number_of_commits showing churn. Files that are both large and frequently changed are the highest-risk architectural hotspots.

Hotspots identified and resolved:

File	Problem	Fix	LOC Before	LOC After
blueprints.service.ts	Single Responsibility Violation — CRUD +	Extracted BacktestService as	471	139

File	Problem	Fix	LOC Before	LOC After
	backtest engine in one class	separate NestJS provider		
alpaca.adapter.ts	God Class — HTTP fetching, SDK trading, and response mapping in one file	Extracted AlpacaMarketDataClient for pure HTTP fetching	473	197
BacktestPanel.tsx / BacktestPreviewPanel.tsx	DRY Violation — shared components copy-pasted	Extracted EquityCurve, TradeRow, backtest-formatters.ts to shared location	265	177
BlueprintReviewTable.tsx	High churn from inline state management in admin page	Extracted useAdmin hook to absorb state management and API calls	64 (churn)	Stable

5. Database Design

5.1 Schema Overview

The database uses PostgreSQL managed through Prisma 5. The schema consists of five models: User, Blueprint, Subscription, ApiKey, and TradeLog.

User

- id, email, password (bcrypt hashed), role (TESTER | PROVIDER | ADMIN)
- Relations: apiKeys[], subscriptions[], blueprints[]

Blueprint

- id, title, description, parameters (Json), isVerified
- Relations: authorId → User, subscriptions[]

Subscription

- id, isActive
- symbolOverride String? — tester-chosen asset; overrides blueprint symbol
- Relations: userId → User, blueprintId → Blueprint, tradeLogs[]

ApiKey

- id, encryptedKey, encryptedSecret (AES-256-GCM)
- label String @default('default')
- Unique constraint: @@unique([userId, label])

TradeLog

- id, symbol, side (TradeSide enum: buy | sell | hold)
- price, quantity, pnl Float?, status String
- executedAt (append-only — no updatedAt)
- Relations: subscriptionId → Subscription

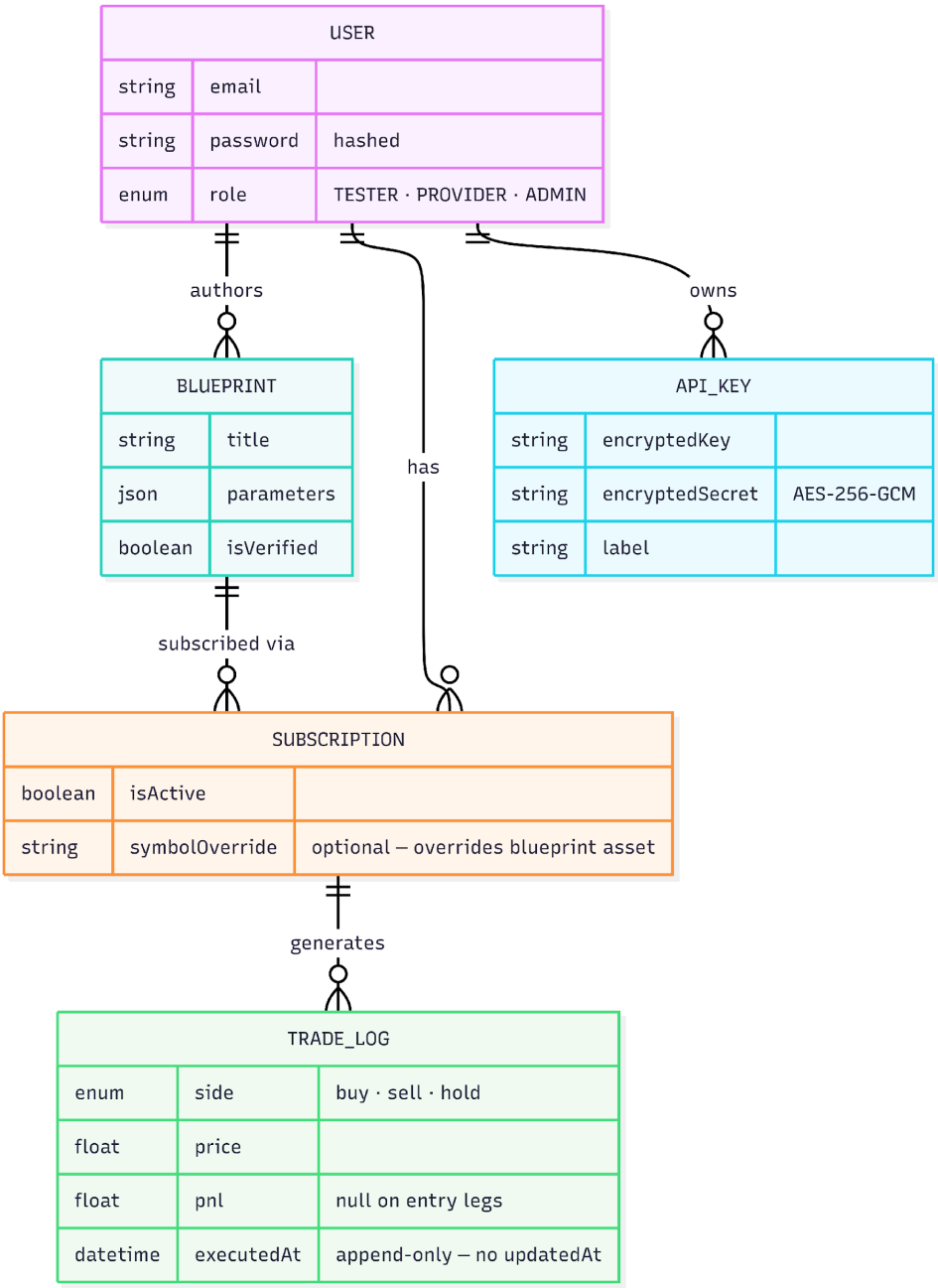

Figure 5 — Database ER Diagram

5.2 Key Design Decisions

Decision	Rationale
TradeSide as Prisma enum	Prevents casing inconsistencies ('BUY' vs 'buy') that break alternation logic
Blueprint.parameters as Json	Flexible enough for multiple strategy types (RSI, ICT); validated by Zod at application layer on every read

Decision	Rationale
@@unique([userId, label]) on ApiKey	Allows multiple broker accounts per user, identified by label
Subscription.symbolOverride nullable	Lets testers target different assets per subscription without altering blueprint parameters; null means use blueprint default
No updatedAt on TradeLog	Enforces append-only semantics — schema itself prevents accidental updates
TradeLog.status as free-text String	Stores rich broker/engine state ('filled', 'bracket_entry:...', 'ict_hold:NO_H1_BIAS') without requiring enum migrations
PrismaModule declared @Global()	PrismaService available to all repositories without explicit imports in each module

6. Role & Permission Structure

Every protected endpoint uses @UseGuards(JwtAuthGuard, RolesGuard) and @Roles(Role.X). Role enforcement is entirely server-side — the client cannot influence its own role. Registration always assigns Role.TESTER; privileged roles are assigned by an admin only.

Role	Permissions
TESTER	Browse marketplace, subscribe to blueprints, manage API keys, view own trade logs
PROVIDER	All TESTER permissions + create, edit, delete own blueprints
ADMIN	All permissions + verify blueprints, view all users and blueprints, assign user roles

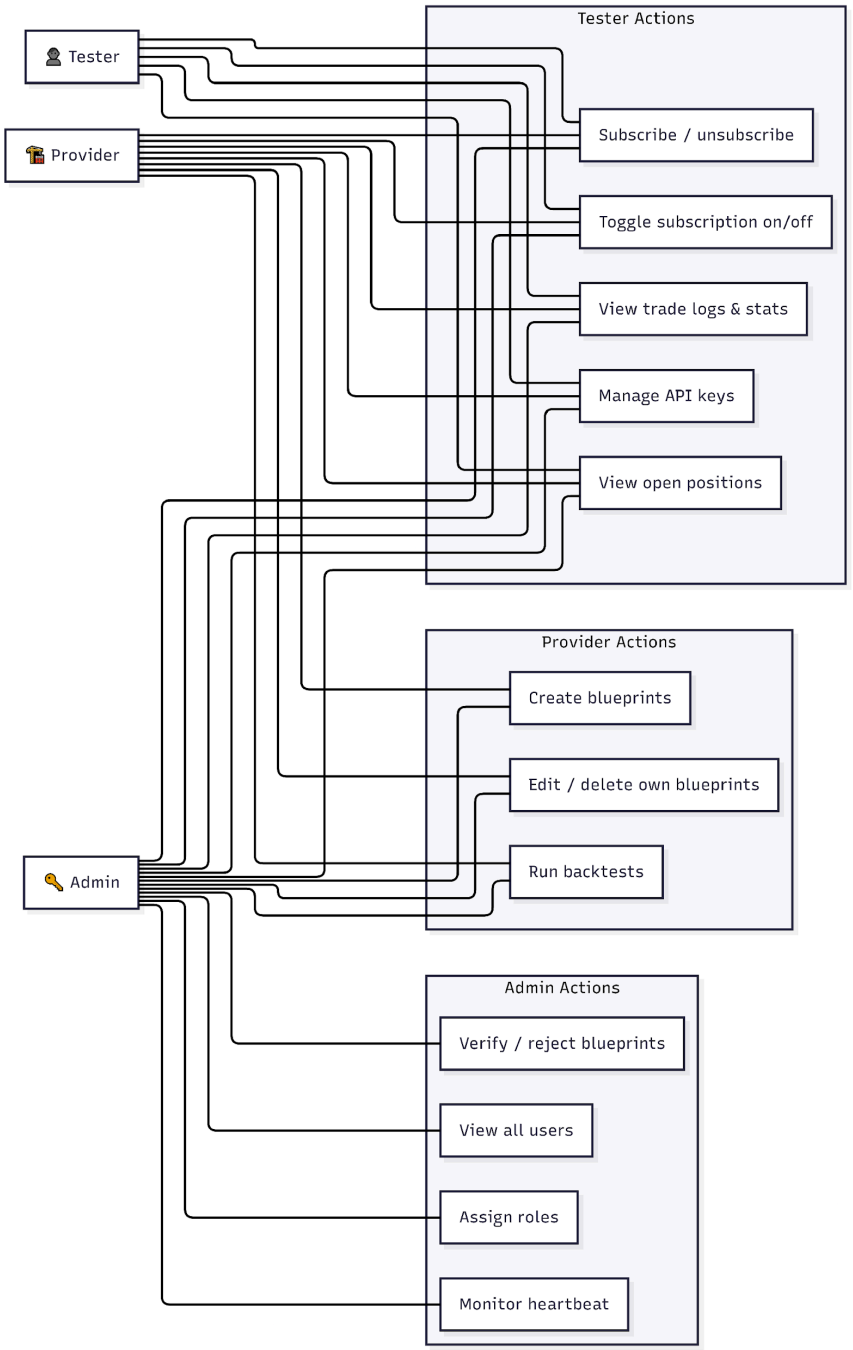

Figure 6 — Role & Permission Map

6.1 Tester API Endpoints

Testers interact with subscriptions, API key management, positions, and blueprint backtests. The Subscriptions, Positions, and API Keys controllers are restricted to Role.TESTER at the class level.

Method	Endpoint	Purpose
GET	/subscriptions	List all subscriptions for the authenticated user
POST	/subscriptions	Create a new subscription to a blueprint
PATCH	/subscriptions/:id/toggle	Toggle a subscription active/inactive
DELETE	/subscriptions/:id	Remove a subscription
GET	/subscriptions/:id/trade-logs	Fetch paginated trade logs for a subscription
GET	/subscriptions/:id/stats	Get statistics (win rate, PnL, trade counts)
GET	/positions	List open positions via the user's Alpaca credentials
GET	/api-keys/status	Check whether the user has an API key configured
GET	/api-keys	List all API keys (with labels) for the user
POST	/api-keys	Create or update an API key (upsert by label)
DELETE	/api-keys	Delete an API key by label
POST	/api-keys/verify	Verify that stored credentials are valid with Alpaca
GET	/blueprints/:id/backtest	Run a backtest against a saved blueprint's parameters
POST	/blueprints/backtest	Run an ad-hoc backtest with arbitrary parameters

6.2 Provider API Endpoints

Providers have all Tester permissions plus the ability to create and manage their own blueprints. Write operations check that the authenticated user is the blueprint author before allowing edits or deletes.

Method	Endpoint	Purpose
POST	/blueprints	Publish a new blueprint (pending admin verification)
GET	/blueprints/my/list	List all blueprints authored by the authenticated provider
PATCH	/blueprints/:id	Update an existing blueprint (author only)
DELETE	/blueprints/:id	Delete a blueprint (author only)
GET	/blueprints/:id/backtest	Run a backtest against a saved blueprint's parameters
POST	/blueprints/backtest	Run an ad-hoc backtest preview with arbitrary parameters

6.3 Admin-Only Endpoints

Method	Endpoint	Purpose
GET	/blueprints/admin/all	View all blueprints including unverified
PATCH	/blueprints/:id/verify	Approve or revoke a blueprint
GET	/auth/users	List all registered users
PATCH	/auth/users/:id/role	Promote or demote a user's role
GET	/heartbeat/status	Monitor system cron execution status

6.4 Public Endpoints (any authenticated user)

Method	Endpoint	Purpose
GET	/blueprints	Browse all verified blueprints (marketplace)
GET	/blueprints/:id	View blueprint details
GET	/blueprints/:id/signal	Get a dry-run signal for a blueprint
GET	/market-data/bars	Fetch OHLCV bars for a symbol
GET	/auth/me	Get the currently authenticated user's profile

RBAC implementation files: auth/roles.guard.ts, auth/roles.decorator.ts, auth/jwt-auth.guard.ts. The decorator pattern keeps role requirements co-located with each route definition, making the security posture auditable in a single file pass.

7. Implementation Details

7.1 Completed Capabilities

Capability	Status
Multi-role user system (PROVIDER / TESTER / ADMIN)	Complete
Admin user management — list users, assign roles	Complete
Blueprint CRUD with admin verification gate	Complete
RSI mean-reversion signal engine — pure functions, fully tested	Complete
ICT / Smart Money Concepts engine — multi-timeframe, 9 pure functions	Complete
Heartbeat auto-execution every 60 seconds	Complete
Per-user encrypted Alpaca API credentials	Complete
Backtest simulation with equity curve, slippage, commission, Sharpe, max drawdown	Complete
Live market bar fetching — stocks and crypto	Complete
Open positions viewer (per-user, credential-delegated)	Complete
Subscription stats with realized + unrealized PnL from live Alpaca positions	Complete
Paginated trade log with server-side fetch	Complete
Tester symbol override — choose which asset to run each strategy on	Complete
Immutable append-only TradeLog audit trail	Complete
Shared Zod contracts between API and Web	Complete
Role-enforced API surface	Complete
Symbol normalization (AAPL, BTC/USD, BTCUSD variants)	Complete
Market hours guard — ET hours for stocks, 24/7 for crypto	Complete
Timeframe boundary alignment (5Min, 15Min, 1Hour, 1Day)	Complete
Smart trade side alternation enforcement	Complete

7.2 Test Coverage

- trading.engine.ts — fully covered with a dedicated spec (RSI, SMA, signal, PnL, all ICT functions)
- heartbeat.service.spec.ts — 25 tests: status, tick behaviour, filledPrice fallback, PnL regression suite for both execution modes
- blueprints.service.ts — backtest and dry-run logic covered with a 332-line spec
- apps/web/src/lib/api-client/base.ts — fetch wrapper covered (Zod validation, error handling, 204 handling)

7.3 Known Issues & Resolutions

Blueprint Parameters as Untyped JSON (Partially Mitigated)

Blueprint.parameters is stored as Json. Out-of-band DB inserts can persist malformed parameters. Current mitigation: the heartbeat service runs BlueprintParametersSchema.safeParse() on every tick. A full fix (normalised BlueprintParameters table) was out of scope.

No Caching on Market Data Fetches (Resolved)

Fixed by adding a Map cache in heartbeat.service.ts keyed on 'symbol:timeframe:limit'. TTLs match the timeframe (1Min=60s, 5Min=5min, 15Min=15min, 1Hour=60min, 1Day=24h). Cache is wiped at the start of each tick.

No API Rate Limiting (Resolved)

Fixed by integrating @nestjs/throttler with two tiers: short (10 req/s burst) and medium (200 req/min sustained), applied globally via ThrottlerGuard.

Backtest Quality Was Optimistic (Resolved)

Fixed by adding slippagePct and commissionPerTrade as optional query parameters, and computing maxDrawdown and sharpeRatio as output fields.

Market Hours Timezone Hardcoded (Resolved)

Fixed by making the timezone configurable via MARKET_TIMEZONE environment variable with 'America/New_York' as the default.

One API Key Per User (Resolved)

Fixed by removing the unique constraint on ApiKey.userId and adding a labeled key system with @@unique([userId, label]). Users can now manage multiple broker accounts identified by label.

No Structured Logging (Resolved)

Fixed by implementing JsonLoggerService that emits newline-delimited JSON log records with timestamp, level, context, and message fields.

PnL Double-Counted on Entry Legs (Resolved)

Fixed by gating P&L recording on isExitLeg rather than lastTradeSide !== null. In BUY_LOW_SELL_HIGH mode, only SELL legs record P&L; in SELL_HIGH_BUY_LOW, only BUY legs do. Entry legs always receive pnl: null.

No User Notifications (Not Resolved)

Testers must poll the trade logs UI to learn about fills, errors, or deactivations. Implementing push notifications requires a new persistence model, a delivery channel (WebSockets, SSE, or email), and frontend integration — out of scope for this iteration.

7.4 Architecture Risk Assessment

Component	Risk Score	Highest Risk Factor
Heartbeat Execution	39 ⚠	Scalability — all subscriptions share one in-process cron loop
Auth / Registration	18	Security — JWT cannot be revoked before expiry
API Key Vault	16	Security — encryption key loss makes all credentials unrecoverable
Subscription Management	15	Data Integrity — subscription toggle state is the sole heartbeat gate
Blueprint Marketplace	14	Availability — unverified blueprint parameters can cause heartbeat skips

Top mitigations identified:

- P0 — Heartbeat scalability: Extract to a dedicated worker process if subscription count exceeds ~100 active
- P0 — External monitoring: Add health-check endpoint and alerting for missed heartbeat ticks
- P1 — JWT revocation: Introduce a short-lived token TTL (15 min) with a refresh token flow

7.5 Architecture Decision Summary

Decision	Pattern Applied	Location
Broker-agnostic trading logic	Hexagonal / Ports & Adapters	IBrokerAdapter + AlpacaAdapter + trading.engine.ts
Database access isolation	Repository Pattern	One *.repository.ts per feature module
HTTP request handling	Thin Controller (validate → service → return)	All *.controller.ts files
Feature separation	NestJS Feature Modules	10 modules, vertical slice structure
Shared type contracts	Single source of truth	packages/types — Zod schemas + interfaces
Broker injection	Dependency Injection (string token)	'IBrokerAdapter' in TradingModule
Role enforcement	RBAC via Guards	JwtAuthGuard + RolesGuard + @Roles()
Background execution	Cron + Promise.allSettled	HeartbeatService
Credential security	AES-256-GCM, in-memory decryption only	EncryptionService + ApiKeysService
Audit trail	Append-only ledger	TradeLog — no update/delete
End-to-end type safety	TypeScript + Zod	DB schema → API → React component
Build efficiency	Monorepo with cache	Turborepo + pnpm workspaces

Decision	Pattern Applied	Location
Architectural health monitoring	CodeCharta fitness function	rawtextparser + gitlogparser → vantrade.cc.json
SRP enforcement	God-class decomposition	BacktestService extracted from BlueprintsService; AlpacaMarketDataClient extracted from AlpacaAdapter
DRY enforcement	Shared component extraction	EquityCurve, TradeRow, backtest-formatters.ts shared across backtest panels